TECHNICAL REPORT & ARCHITECTURE SPECIFICATION

Eidovia Architecture

An AR & Mobile Educational DNA Tutoring System with Microservices, Local LLM Inference, and Administrative Controls

---

Project: Eidovia Thesis Project

Branch: `AI-code` • **Date:** October 2026

Scope: API Gateway, Admin Console, ELT Logger, Ollama LLM, PostgreSQL, Redis

1. System Context Architecture (C4 Level 1)

The Eidovia platform is an interactive educational ecosystem designed for genetics and DNA tutoring. The system context model defines the primary human actors (students interacting via the mobile client and academic administrators operating the management portal) alongside core backend components and external infrastructure.

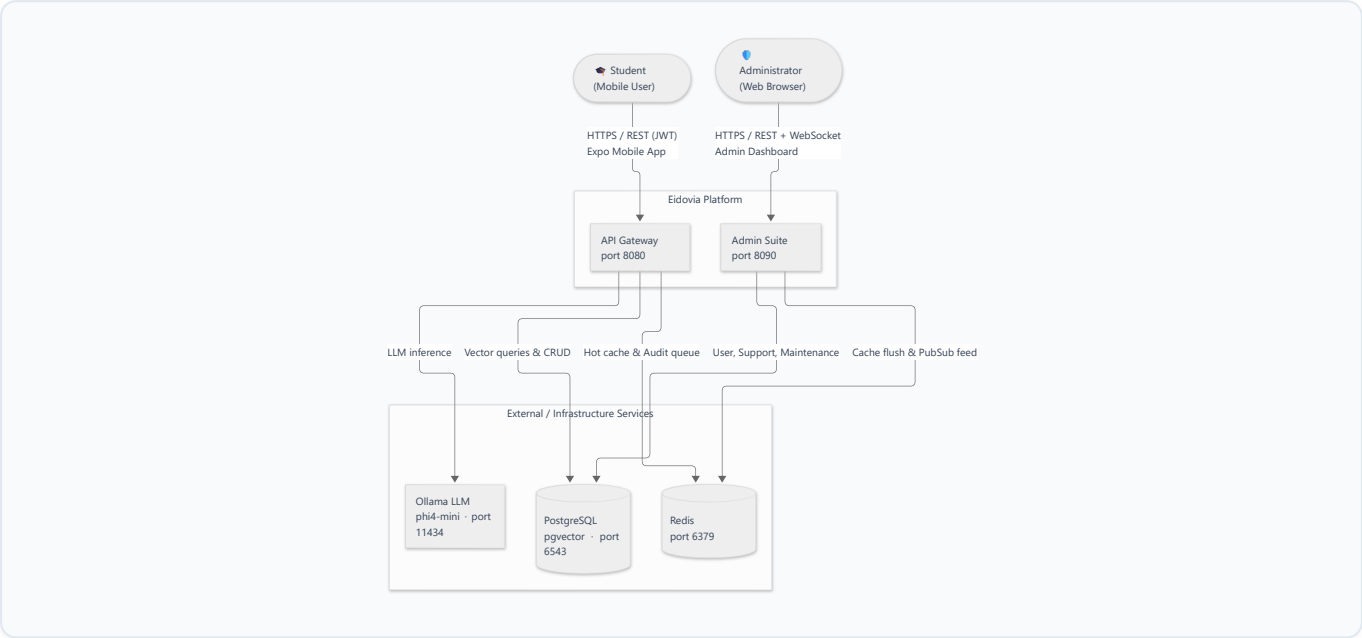


Figure 1: High-level System Context (C4 Level 1) Diagram.

Actor & Boundary Breakdown

Entity	Type	Role & Description
Student	End User	Interacts with AR models and conversational DNA tutor via the React Native / Expo application.
Administrator	Staff User	Oversees system health, manages student accounts, handles support tickets, and monitors audit logs.
API Gateway	Service	Student-facing API gateway routing authentication, curriculum modules, and AI requests.
Admin Suite	Service	Isolated administrative portal on a dedicated port ensuring complete separation of concerns.

2. Container Architecture (C4 Level 2)

The system is deployed using Docker Compose on an isolated container network (`eidovia_net`). Each container fulfills a single responsibility, maintaining modularity and strict boundary enforcement.

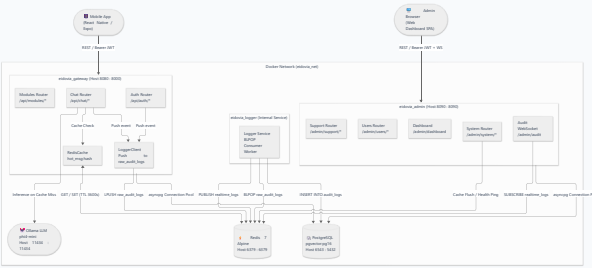


Figure 2: Container Topology (C4 Level 2) with Internal Service Communication.

Container Specifications

Container Name	Base Image	Host Port	Internal Port	Primary Responsibility
`eidovia_gateway`	Python 3.11-slim	8080	8000	Mobile API gateway, student authentication, chat and module endpoints.
`eidovia_admin`	Python 3.11-slim	8090	8090	Autonomous administrative portal, RBAC management, WebSocket telemetry.
`eidovia_logger`	Python 3.11-slim	—	Internal	Asynchronous ELT event consumer; uncouples logging I/O from request paths.
`eidovia_ollama`	ollama/ollama	11434	11434	Local LLM inference executing `phi4-mini` DNA tutor model.
`eidovia_postgres`	pgvector:pg16	6543	5432	Primary relational database with vector similarity extension.
`eidovia_redis`	redis:7-alpine	6379	6379	Hot message response cache, audit FIFO queue, and live pub/sub channels.

3. Asynchronous Audit Logging & Real-Time Event Pipeline

To preserve high throughput across mobile endpoints, audit logging employs an asynchronous ELT (Extract, Load, Transform) queue pattern. Gateway requests push event records to Redis in non-blocking fashion, which an independent worker ingests and persists to PostgreSQL before broadcasting to active administrator sessions.

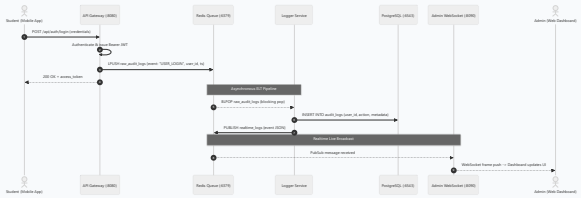


Figure 3: End-to-end Audit Log Event Flow and Real-time Broadcast.

> **Zero-Latency Guarantee:** The gateway performs only a single in-memory `LPUSH` operation (~0.5ms). Database writes are entirely offloaded to `eidovia_logger`, preventing database contention from slowing mobile student interactions.

4. Chat & LLM Hot-Cache Acceleration Pipeline

To reduce latency and computational load during repeated student inquiries on foundational DNA topics, the gateway incorporates a SHA-256 hashed hot message cache in Redis with an automated 1-hour TTL eviction policy.

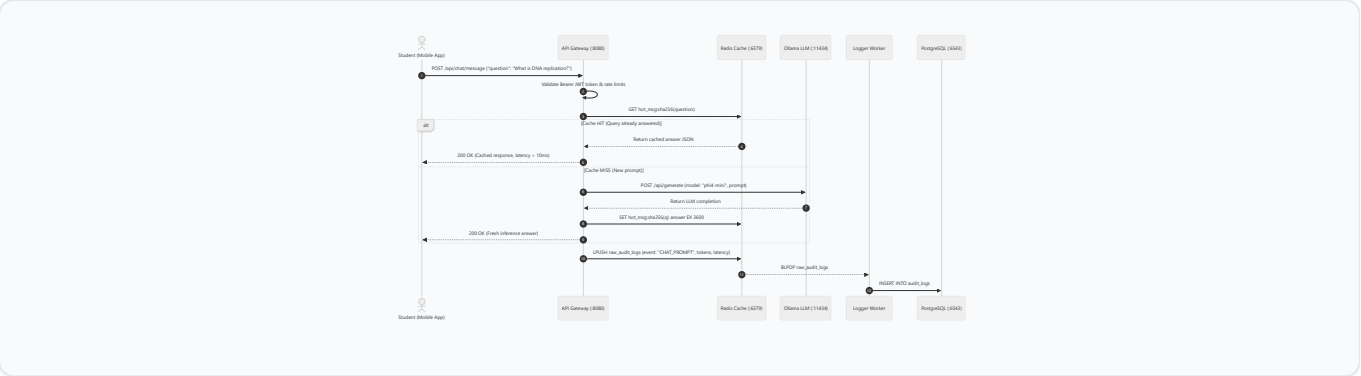


Figure 4: Chat Interaction with Hot-Message Cache Acceleration.

Cache Strategy Characteristics

- **Key Generation:** Deterministic hashing using `hot_msg:sha256(normalized_question)`.
- **Cache Invalidation:** Global cache flush endpoint available to administrators via `POST /admin/system/cache/flush`.
- **Off-Topic Safeguard:** The system prompt enforces DNA and genetics curriculum boundaries, returning standard refusals for non-domain inquiries.

5. Admin Authentication & Role-Based Access Control

To satisfy enterprise security standards, the administration center (`apps/admin`) runs as a completely independent service on port 8090 with strict role validation (`is_admin = TRUE`) and dedicated session token management.

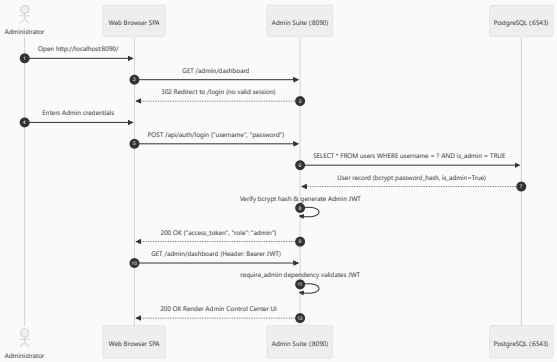


Figure 5: Administrator Authentication and Role-Based Authorization Flow.

> **Security Boundary:** Standard student credentials cannot authenticate against the Admin Suite, as the SQL lookup explicitly mandates `WHERE is_admin = TRUE`. Even if a valid student token is presented, the `require_admin` dependency denies access.

6. Host Deployment & Infrastructure Topology

The entire backend ecosystem is provisioned via Docker Compose under Windows Hyper-V. Due to Hyper-V dynamic port exclusions reserving host ports 7904–8003, the API Gateway is mapped to host port 8080.

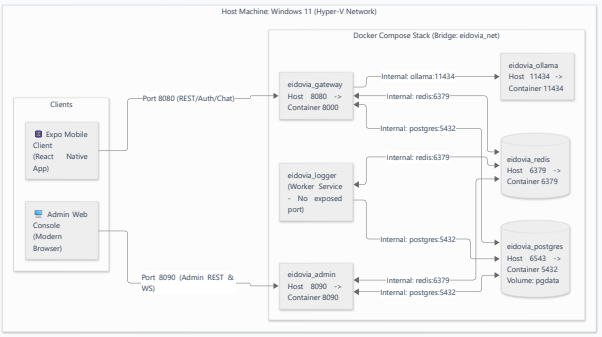


Figure 6: Host Machine Deployment Topology and Network Bridging.

Port Mapping & Exclusion Handling

Service	Container Port	Host Binding	Rationale & Notes
Student API Gateway	8000	8080	Relocated from 8000 to avoid Windows Hyper-V reserved range (7904–8003).
Admin Center	8090	8090	Dedicated administrative listener accessible via local web browser.
PostgreSQL	5432	6543	Prevent collision with host system PostgreSQL installations.
Redis	6379	6379	In-memory data store accessible for debug telemetry and caching.
Ollama	11434	11434	Local LLM REST endpoint executing quantized DNA inference models.

7. Database Entity-Relationship (ER) Architecture

The persistent data model is managed in PostgreSQL with the `pgvector` extension. Seven core relational tables provide normalized storage for identity, authorization, support, audit logging, and curricular embeddings.

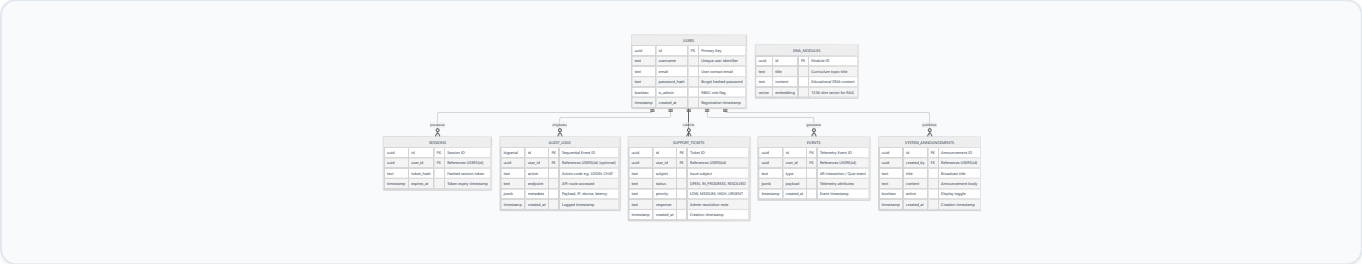


Figure 7: Relational & Vector Database Entity-Relationship (ER) Schema.

Key Schema Characteristics

- **Auditing:** `AUDIT_LOGS` uses an unconstrained `jsonb` metadata field to capture request payloads, IP origins, and model token usage without schema rigidness.
- **Support Workflow:** `SUPPORT_TICKETS` enables bi-directional communication between students and administrators.
- **Curriculum Vectors:** `DNA_MODULES` integrates vector similarity indexing via `pgvector` for future Retrieval-Augmented Generation (RAG).